Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes
Département Ingénierie des Systèmes Embarqués
Filière Smart Systems Engineering

TP 4 : Un dataset et un premier modèle audio pour surveiller le moteur du tapis

Projet fédérateur EcoTri 4.0, sous-projet SP4 « Écouter la machine » et jalon J4

Pr.Hassan BERBIA/Pr.Aissam BERRAHOU

2 novembre 2026

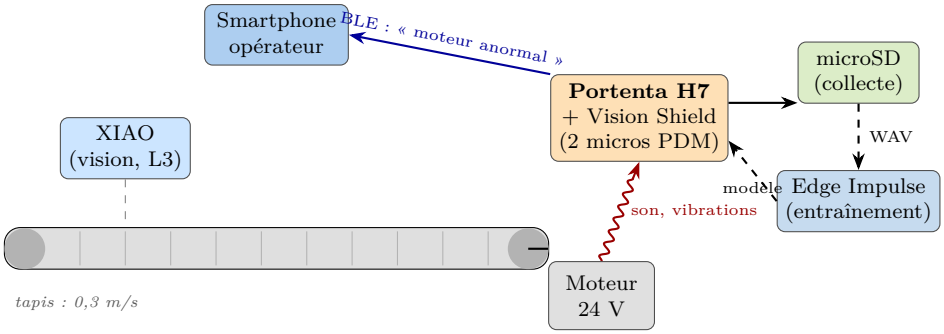
Objectifs du TP

Le Tutorial 4 a donné à la Portenta une oreille et un premier détecteur d'anomalie statistique. Dans ce TP, chaque équipe (3 ou 4 étudiants) construit un **dataset de qualité** pour trois états du moteur (*normal*, *anormal*, *arrêt*), mesure l'effet d'une **fuite de données**, puis entraîne dans **Edge Impulse** un modèle qui combine un classifieur et un bloc de détection d'anomalies. Le modèle est évalué honnêtement, sur une session jamais vue, et comparé à la référence embarquée du tutoriel. La fiche du dataset et le premier modèle constituent le **jalon J4**.
À la fin de ce TP, vous serez capable de :

- Rédiger et suivre un plan de collecte par sessions, avec des réglages figés et des métadonnées.
- Vérifier un dataset audio : durées, niveaux, saturation, équilibre des classes.
- Mettre en évidence la fuite de données en comparant découpage aléatoire et découpage par session.
- Configurer un impulse Edge Impulse (fenêtrage, MFE, classifieur, K-means) et en interpréter les sorties.
- Choisir un seuil d'anomalie sur la validation et l'évaluer sur le test.
- Documenter le dataset par une fiche (*Datasheets for Datasets*).

Livrable du jalon J4 (à déposer en fin de lecture) : fiche du dataset, dataset et métadonnées, résultats et analyse (partie 6).

1 Contexte : surveiller le moteur sans panne étiquetée



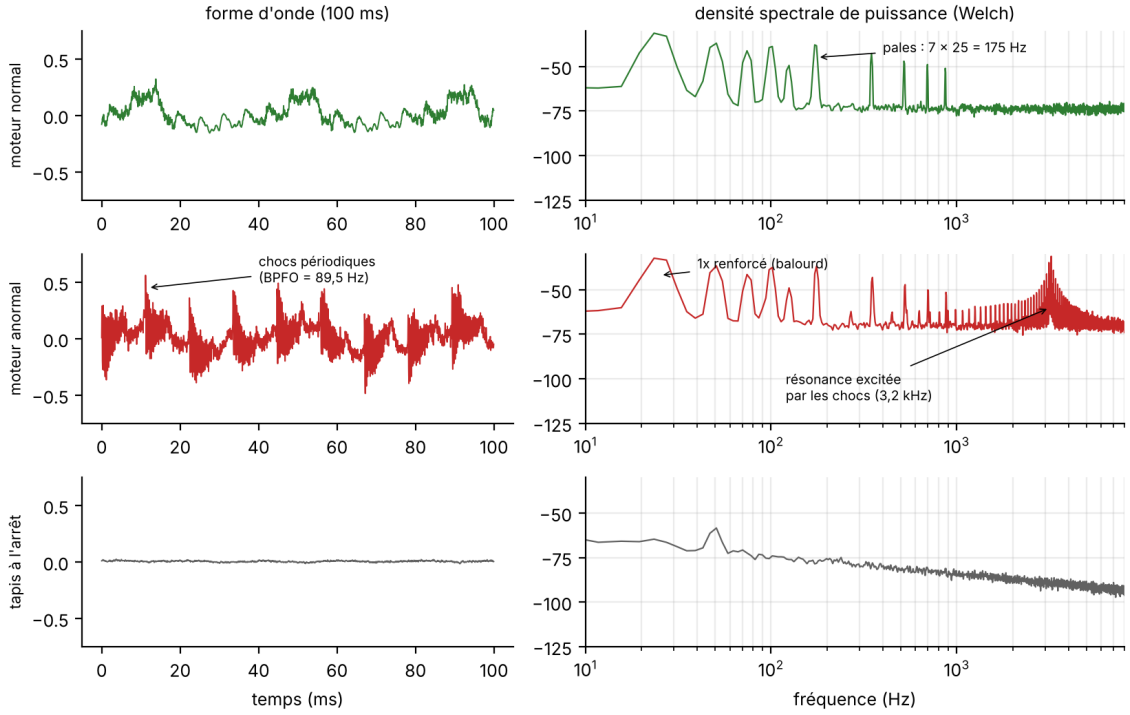
Le sous-projet dans la station : collecte sur microSD, entraînement dans Edge Impulse, puis alerte vers le smartphone (lecture 3). Le déploiement sur la carte sera finalisé en lecture 6.

Le centre de tri ne possède aucun enregistrement de panne réelle. On simule donc deux défauts réalistes et réversibles sur le moteur ou le ventilateur de la maquette :

— **balourd** : une petite masse (pâte à fixe, 1 à 2 g) collée sur une pale ou sur la poulie ;

— **frottement** : une languette de papier ou de plastique fin qui effleure une pièce en rotation.

La classe *anormal* regroupe les deux défauts ; vous garderez l'information du type de défaut dans les métadonnées. La classe *arrêt* est le bruit de l'atelier moteur coupé.



Signatures attendues (signaux simulés du cours) : raies de rotation et de pales pour le moteur normal, raie 1x renforcée et chocs pour l'anormal, bruit diffus à l'arrêt.

Astuce

Pas de moteur disponible ? Utilisez un ventilateur USB, ou jouez près de la carte des enregistrements du jeu public **MIMII** (Hitachi, ventilateurs et pompes, normal et anormal) avec un haut-parleur fixe. Chaque « session » correspond alors à un identifiant de machine du jeu de données.

2 Pré-requis

Matériel (par équipe)

- **Portenta H7** + **Vision Shield** (Ethernet) + carte **microSD**, câble USB-C.
- Le moteur de la maquette (ou un ventilateur USB), pâte à fixe, languettes de papier, ruban adhésif pour marquer les positions de la carte.

Logiciel

1. **OpenMV IDE** et les scripts du Tutorial 4 (`bandes.py`, `audio_wav.py`).
2. **Python 3** avec `numpy`, `scipy`, `scikit-learn`, `matplotlib` et `pandas`.
3. Un compte gratuit **Edge Impulse** (studio.edgeimpulse.com) ; optionnellement l'outil en ligne de commande `edge-impulse-uploader` (Node.js).

3 Étapes détaillées du TP

Partie 1 : Le plan de collecte (15 min)

Avant d'enregistrer la moindre seconde, l'équipe rédige son plan. Le principe, vu en cours : chaque **session** réunit les trois classes enregistrées **en alternance**, dans les mêmes conditions ; les sessions diffèrent entre elles par au moins un facteur (position de la carte, vitesse, moment, bruit ambiant).

Session	Conditions (à compléter)	Rôle	Contenu
s1	position A, vitesse nominale	entraînement	3 classes × 2 min, alternées
s2	position B (à 20 cm de A)	entraînement	idem
s3	position A, ventilation de la salle allumée	validation	idem
s4	position C, autre moment	**test** (non regardé avant la partie 5)	idem

Plan de collecte minimal. Fixez et notez : $f_e = 16$ kHz, gain (étape 2 du tutoriel), filtre passe-haut, durée des fichiers (2 s).

Question

Pourquoi alterner les classes à l'intérieur d'une session (normal, anormal, arrêt, normal…) plutôt que d'enregistrer toutes les fenêtres « normal » puis toutes les fenêtres « anormal » ? Citez un facteur qui pourrait varier pendant la session et créer un biais.

Partie 2 : La collecte (35 min)

On étend le script du tutoriel : le bouton de la carte n'existe pas sur la Portenta, on pilote donc la classe depuis l'IDE en relançant le script avec `CLASSE` modifiée, et l'on ajoute un fichier `metadata.csv` qui trace chaque enregistrement.

```
# collecte_tp4.py - ajouter à audio_wav.py (Tutorial 4) après l'écriture de chaque
    fichier
DEFAUT = "balourd"          # "" pour normal et arret ; "balourd" ou "frottement"
    pour anormal
POSITION = "A"
OPERATEUR = "equipe3"

def ligne_meta(nom, x):
    rms = np.sqrt(np.mean(x * x * 1.0)); crete = int(np.max(abs(x)))
    return "%s,%s,%d,%s,%s,%d,%d,%.1f,%d,%s\n" % (
        nom.split("/")[-1], CLASSE, SESSION, DEFAUT, POSITION, FE, GAIN_DB,
        rms, crete, OPERATEUR)

META = DOSSIER + "/metadata.csv"
if "metadata.csv" not in os.listdir(DOSSIER):
    with open(META, "w") as f:
        f.write("fichier,classe,session,defaut,position,fe,gain_db,rms,crete,
    operateur\n")
# ... dans la boucle, après f.write(tampon) :
#     with open(META, "a") as m: m.write(ligne_meta(nom, x))
```

1. Pour chaque session, enregistrez par blocs de 10 fichiers de 2 s, en alternant : `normal`, `anormal` (balourd), `arret`, `normal`, `anormal` (frottement), `arret`. Vous obtenez environ 2 minutes par classe et par session.
2. Ne modifiez jamais le gain ni la position **à l'intérieur** d'une session.
3. Notez sur papier tout événement (porte qui claque, voix) avec le nom du fichier concerné.
4. Copiez le dossier `ecotri` sur le PC.

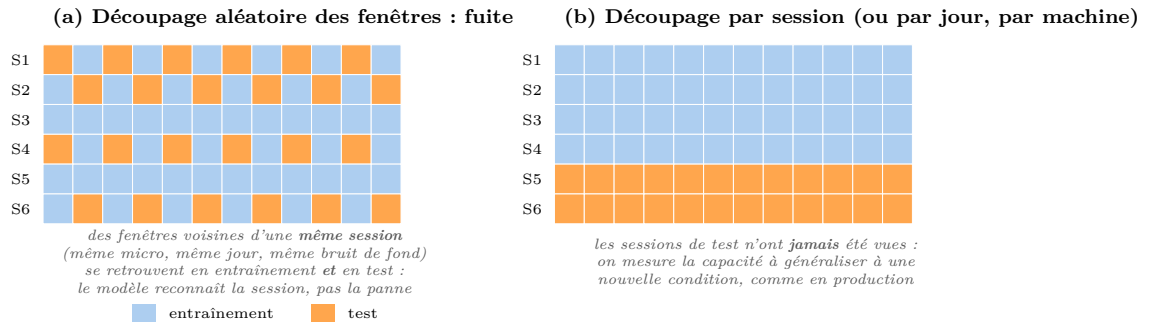
```
# verif_dataset.py - sur le PC : contrôle du dataset
import pandas as pd, numpy as np
from scipy.io import wavfile
m = pd.read_csv("ecotri/metadata.csv")
for f in m.fichier:
    fe, x = wavfile.read("ecotri/" + f)
    assert fe == 16000 and len(x) == 32000, f
print(m.groupby(["session", "classe"]).size().unstack())          # équilibre
print(m.groupby("classe")[["rms", "crete"]].describe().round(0))   # niveaux
print("fichiers saturés :", list(m[m.crete >= 32700].fichier))
```

Question

Le RMS moyen de la classe « arrêt » est-il nettement inférieur aux deux autres ? Le RMS de « anormal » diffère-t-il de celui de « normal » ? Si oui, un modèle pourrait-il se contenter du niveau sonore ? Est-ce souhaitable, sachant que la carte peut être déplacée ?

Partie 3 : Mesurer la fuite de données (25 min)

Avant Edge Impulse, on quantifie sur le PC l'effet du découpage. Le script calcule les MFE (40 bandes) de chaque fenêtre d'une seconde, résume chaque fenêtre par la moyenne et l'écart-type des bandes, puis compare un découpage aléatoire des fenêtres et une validation croisée **par session**.



Rappel du cours : découpage aléatoire des fenêtres (fuite) contre découpage par session.

```python
# fuite_tp4.py - sur le PC : découpage aléatoire contre découpage par session
import numpy as np, pandas as pd
from scipy.io import wavfile
from scipy import signal
from sklearn.model_selection import train_test_split, GroupKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier

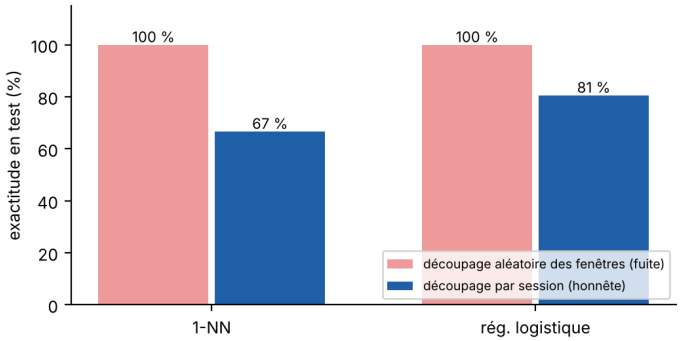
def banc_mel(nf=40, nfft=512, fe=16000):
    mel = lambda f: 2595 * np.log10(1 + f / 700); hz = lambda m: 700 * (10 ** (m /
    2595) - 1)
    b = np.floor((nfft + 1) * hz(np.linspace(0, mel(fe / 2), nf + 2)) / fe).astype(
    int)
    H = np.zeros((nf, nfft // 2 + 1))
    for i in range(nf):
        H[i, b[i]:b[i + 1]] = np.linspace(0, 1, b[i + 1] - b[i], endpoint=False)
        H[i, b[i + 1]:b[i + 2]] = np.linspace(1, 0, b[i + 2] - b[i + 1], endpoint=
    False)
    return H
H = banc_mel()

def mfe(x):                                          # trames 25 ms, pas 10 ms, Hann, FFT
    512
    _, _, Z = signal.stft(x, 16000, "hann", nperseg=400, noverlap=240, nfft=512,
                          boundary=None, padded=False)
    return np.log10(H @ np.abs(Z) ** 2 + 1e-10)

m = pd.read_csv("ecotri/metadata.csv")
X, y, g = [], [], []
for _, r in m.iterrows():
    _, x = wavfile.read("ecotri/" + r.fichier)
    for k in range(0, len(x) - 16000 + 1, 8000):    # fenêtres de 1 s, pas de 0,5 s
        M = mfe(x[k:k + 16000].astype(float))
        X.append(np.r_[M.mean(1), M.std(1)]); y.append(r.classe); g.append(r.session)
X, y, g = np.array(X), np.array(y), np.array(g)

for nom, clf in [("1-NN", KNeighborsClassifier(1)),
                 ("régression logistique", LogisticRegression(max_iter=3000))]:
    p = make_pipeline(StandardScaler(), clf)
    Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.2, random_state=0,
    stratify=y)
    a = p.fit(Xtr, ytr).score(Xte, yte)
    s = cross_val_score(p, X, y, groups=g, cv=GroupKFold(n_splits=len(set(g))))
```

```
    print("%-22s aléatoire : %.1f %%   par session : %.1f %% (+/- %.1f)" %
          (nom, 100 * a, 100 * s.mean(), 100 * s.std()))
```



Le résultat obtenu en cours sur des données simulées. Obtenez-vous un écart du même ordre sur vos propres données ?

Question

1. Quel écart observez-vous entre les deux découpages ? Lequel des deux modèles profite le plus de la fuite, et pourquoi ?
2. Ajoutez au script une expérience où la session de **test** est enregistrée à une position jamais vue : la performance chute-t-elle ? Qu'en concluez-vous sur la fiche du dataset (rubrique « Usages » et « limites connues ») ?

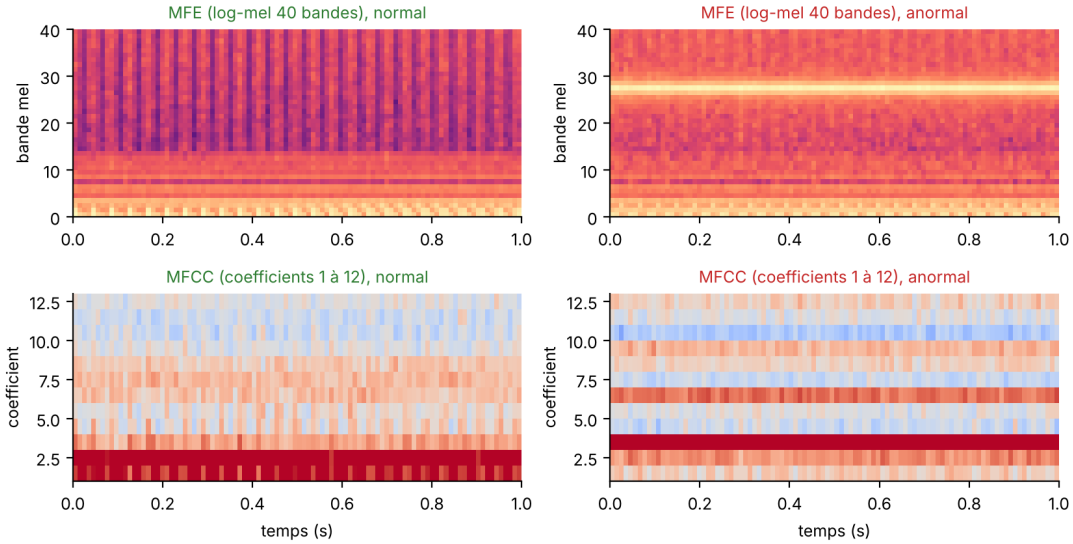
Partie 4 : Le modèle Edge Impulse (35 min)

a) Import des données. Créez un projet `EcoTri-Moteur-equipeN`. Dans **Data acquisition** → **Upload data**, importez les fichiers des sessions 1 à 3 en **Training** et ceux de la session 4 en **Testing**, en indiquant le label (la classe). **Ne laissez pas** Edge Impulse répartir automatiquement entraînement et test : il découperait au hasard. En ligne de commande :

```
edge-impulse-uploader --category training --label normal  ecotri/normal_s[123]_*.wav
edge-impulse-uploader --category training --label anormal ecotri/anormal_s[123]_*.wav
edge-impulse-uploader --category training --label arret   ecotri/arret_s[123]_*.wav
edge-impulse-uploader --category testing  --label normal  ecotri/normal_s4_*.wav
# ... idem pour anormal et arret en session 4
```

b) L'impulse. Dans **Create impulse** :

Bloc	Réglage	Lien avec le cours
Time series data	fenêtre 1000 ms, pas 500 ms, 16 kHz	99 trames ; recouvrement de 50 %
Processing : **Audio (MFE)**	trame 0,025 s, pas 0,01 s, 40 filtres, FFT 512, plancher −52 dB	sons non vocaux : MFE plutôt que MFCC
Learning : **Classification**	3 classes	CNN sur l'image 99 × 40
Learning : **Anomaly detection (K-means)**	3 groupes, caractéristiques sélectionnées	apprend la forme du normal

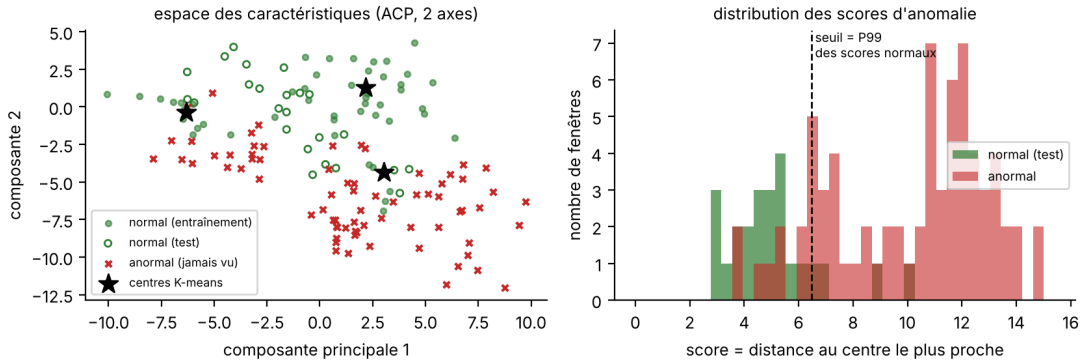


Les MFE que calcule le bloc Audio (MFE), comparées aux MFCC. L'onglet *MFE* d'Edge Impulse affiche la même image pour chaque fenêtre : vérifiez que les classes y sont visuellement différentes.

c) Caractéristiques. Dans l'onglet **MFE**, cliquez sur **Generate features** et observez le *Feature explorer*. Relevez l'estimation de temps de calcul et de RAM du DSP sur la cible (choisissez *Arduino Portenta H7 (Cortex-M7 480 MHz)* en haut de page).

d) Classifieur. Entraînez avec 100 cycles, taux d'apprentissage 0,005, et l'architecture proposée par défaut (convolutions 1D). Pour éviter que l'outil choisisse sa validation au hasard parmi des fenêtres voisines, utilisez la session 3 comme validation si votre version le permet (option de découpage par métadonnées), sinon notez cette limite dans votre analyse.

e) Anomalie. Dans **Anomaly detection**, cliquez sur **Select suggested axes** puis entraînez avec 3 groupes. Notez que ce bloc n'utilise que les fenêtres de **training** : idéalement, il ne devrait voir que du **normal**. Refaites l'entraînement en désactivant (*Disable*) les fichiers « anormal » de l'entraînement, et comparez.



Rappel du cours : un K-means appris sur le normal, et la distribution des scores. Le seuil se choisit sur la validation, d'après le coût des erreurs.

f) Test. Dans **Model testing**, cliquez sur **Classify all** : c'est la session 4, jamais vue. Réglez le seuil d'anomalie (menu *Set confidence thresholds*) à partir des scores de la session 3 (validation), pas de la session 4.

Question

1. Comparez l'exactitude de validation affichée pendant l'entraînement et celle du test sur la session 4. Comment expliquez-vous l'écart ?
2. Quelles fenêtres sont mal classées ? Écoutez-en trois : l'étiquette est-elle juste ? La fenêtre contient-elle un événement extérieur ?
3. Les fenêtres de frottement et de balourd ont-elles le même score d'anomalie ? Lequel des deux défauts le classifieur reconnaît-il le mieux ?

Partie 5 : Comparer à la référence embarquée (15 min)

Rejouez la session 4 devant la carte (ou rechargez ses fichiers WAV dans le détecteur du Tutorial 4 en remplaçant l'acquisition par une lecture de fichier) et comparez les deux approches.

Critère (session 4)	**Détecteur statistique (Tutorial 4)**	**Edge Impulse (MFE + CNN + K-means)**
Rappel des anomalies (%)		
Fausses alarmes sur le normal (%)		
Comportement sur « arrêt »		
Nomme le type d'état ?	non	
Taille du modèle (Ko)	< 1	
RAM estimée sur la Portenta (Ko)		
Temps de calcul par fenêtre (ms)		

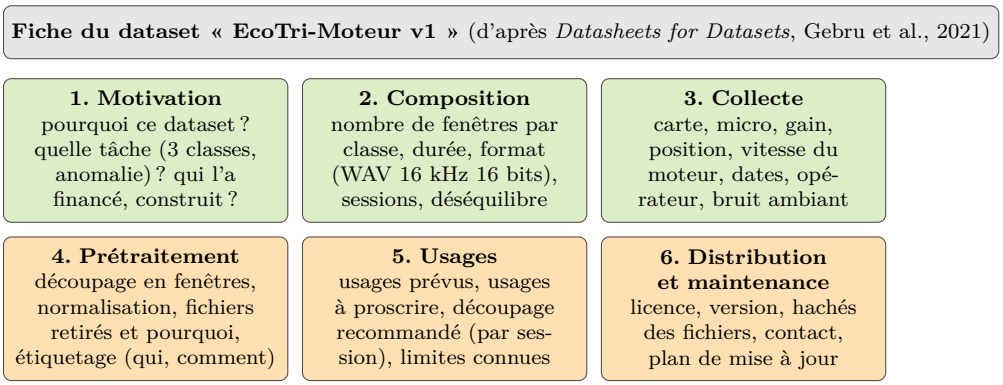
Question

Le gain de performance du modèle appris justifie-t-il son coût (mémoire, temps, effort de collecte) ? Dans quelle situation la référence statistique serait-elle suffisante ? Proposez la logique de décision qui combine classifieur et détecteur d'anomalie (cours, section « Combiner les deux

approches »).

Partie 6 : Livrable du jalon J4 (15 min)

a) Fiche du dataset (deux pages au plus), structurée selon la figure ci-dessous. Chaque rubrique répond aux questions du cours ; la rubrique « Composition » contient le tableau des effectifs par classe et par session.



Structure imposée de la fiche du dataset « EcoTri-Moteur v1 ».

b) Dataset : dossier `ecotri/` (WAV + `metadata.csv`) déposé sur l'espace du cours, nommé `EcoTri-Moteur_equipeN_v1`.

c) Tableau des résultats

Mesure	Valeur	Conditions
Nombre de fenêtres par classe (entraînement / test)		1 s, pas 0,5 s
Exactitude, découpage aléatoire (partie 3)		régression logistique
Exactitude, découpage par session (partie 3)		GroupKFold
Exactitude équilibrée du classifieur EI (session 4)		
Rappel « anormal » / fausses alarmes (session 4)		seuil choisi sur s3
RAM et Flash estimées (DSP + modèles)		Portenta H7, INT8
Latence estimée (DSP + inférence)		Portenta H7

d) Analyse (une page) : écart dû à la fuite, erreurs écoutées et leur cause, choix et justification du seuil, limites connues du dataset et du modèle, et ce que vous collecteriez en priorité pour une version 2.

4 Grille d'évaluation du jalon J4

Critère	Attendu	Points
Plan et collecte	sessions alternées, réglages figés, métadonnées complètes, contrôle du dataset	4
Fiche du dataset	les six rubriques renseignées, limites connues explicites	5
Fuite de données	expérience menée, écart quantifié et expliqué	3
Modèle et évaluation	impulse justifié, test sur session jamais vue, seuil choisi sur la validation	5
Analyse critique	comparaison avec la référence embarquée, erreurs écoutées, version 2 proposée	3
Total		**20**

5 Ressources

— Edge Impulse, documentation des blocs *Audio (MFE)*, *Classification* et *Anomaly detection (K-means / GMM)* : docs.edgeimpulse.com.

— OpenMV, modules `audio` et `ulab` : docs.openmv.io.

— H. Purohit et al., « MIMII Dataset : Sound Dataset for Malfunctioning Industrial Machine Investigation and Inspection », DCASE Workshop, 2019 (zenodo.org/records/3384388).

— T. Gebru et al., « Datasheets for Datasets », *Communications of the ACM*, 64(12), 2021.

— S. Kapoor, A. Narayanan (Princeton), « Leakage and the Reproducibility Crisis in Machine-Learning-Based Science », *Patterns*, 2023.

— scikit-learn, guide « Cross-validation : evaluating estimator performance », section *Group k-fold*.